

TP Guidé : Architecture Distribuée, Réplication et Gestion des Pannes avec Apache Cassandra et l'API Vélib'

Ce TP a pour objectifs de déployer un cluster Cassandra de 3 nœuds en local via Docker, de l'interfacer avec une source de données réelle (l'API Open Data Vélib' de Paris), de configurer un facteur de réplication ($RF = 3$), de manipuler les niveaux de cohérence (**ONE**, **QUORUM**, **ALL**), et de simuler des pannes de nœuds pour observer la tolérance aux pannes.

Étape 1 : Mise en place de l'infrastructure Docker

Créez un dossier de travail (par exemple **tp-cassandra-velib**) et placez-y un fichier **docker-compose.yml** contenant la configuration suivante :

(voir le fichier **docker-compose.yml** dans le depot ZIP)

Lancer **cass1** et **cass2** simultanément
docker compose up -d cass1 cass2

Patiencez un court instant

Laissez les deux conteneurs s'initialiser et s'enregistrer mutuellement (comptez environ 60 à 120 secondes). Vous pouvez suivre leurs logs si besoin avec **docker logs -f cass1**

Vérifier l'état du cluster

Attendez un petit instant que tout le monde se synchronise, puis vérifiez que les 2 nœuds sont bien opérationnels (**UN**) :

docker exec -it cass1 nodetool status

Lancer le troisième nœud **cass3**

docker compose up -d cass3

Vérifier l'état du cluster

Une fois que les deux premiers tournent bien :

```
(1 rows)
root@vm13420237:~/TP_cassandra/script# docker exec -it cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load       Tokens     Owns (effective)  Host ID                               Rack
UN  172.18.0.4    207.9 KiB   16         100.0%            c4ac5523-d1d8-41c4-9d7f-95426bac491f  rack3
UN  172.18.0.3    80.93 KiB   16         100.0%            0a78ae23-ae6d-4023-8182-575223ac7995  rack1
UN  172.18.0.2    85.92 KiB   16         100.0%            82c3ed43-7116-4370-a634-24ae0bcf6f73  rack2
```

Les 3 nœuds ont bien démarré, sont marqués comme **UN** (*Up and Normal*), et se partagent la charge sur le datacenter **dc1**

Étape 2 : Modélisation des données et création du Keyspace ($RF = 3$)

Entrez dans le shell CQL du premier nœud :

```
docker exec -it cass1 cqlsh
```

Créez le keyspace en configurant un facteur de réplication de **3** ($RF = 3$) sur notre unique datacenter **dc1**

```
CREATE KEYSPACE velib_cluster  
WITH replication = {  
    'class': 'NetworkTopologyStrategy',  
    'dc1': 3  
};
```

-- Activez le keyspace pour la session

```
USE velib_cluster;
```

Créez la table pour stocker les disponibilités des stations Vélib' en temps réel :

```
CREATE TABLE stations_velib (  
    station_id text,  
    nom_station text,  
    capacite int,  
    vatiques_disponibles int,  
    bornes_disponibles int,  
    derniere_mise_a_jour timestamp,  
    PRIMARY KEY (station_id)  
);
```

Étape 3 : Alimentation des données depuis l'API Vélib'

Pour injecter de vraies données, nous allons utiliser un petit script Python en local qui interroge l'API Open Data de Paris

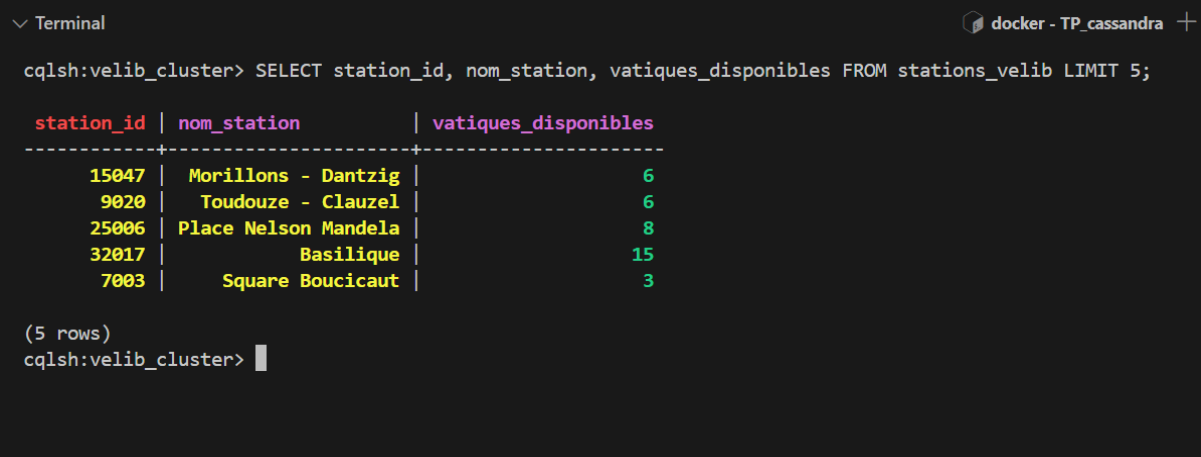
1-Installez le client Python Cassandra sur votre machine hôte :

```
pip install cassandra-driver requests
```

(voir le script `import_velib.py` dans le dossier ZIP)

Vérifiez dans **cqlsh** que les données sont bien présentes :

SELECT station_id, nom_station, vatiques_disponibles FROM stations_velib LIMIT 5;



```

Terminal
cqlsh:velib_cluster> SELECT station_id, nom_station, vatiques_disponibles FROM stations_velib LIMIT 5;

station_id | nom_station | vatiques_disponibles
-----+-----+-----
15047 | Morillons - Dantzig | 6
9020 | Toudouze - Clauzel | 6
25006 | Place Nelson Mandela | 8
32017 | Basilique | 15
7003 | Square Boucicaut | 3

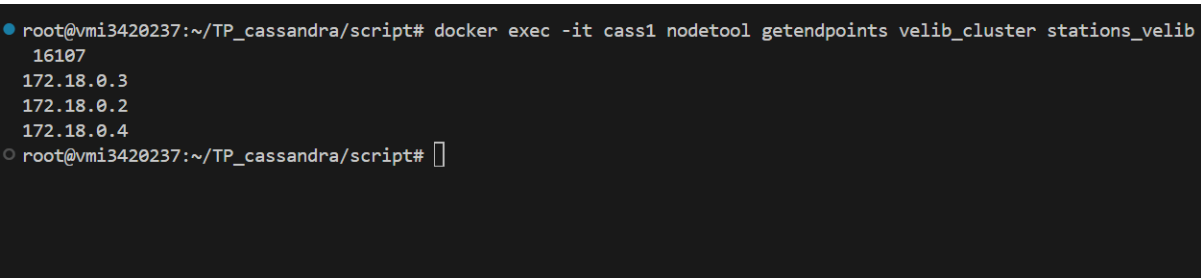
(5 rows)
cqlsh:velib_cluster>
  
```

Étape 4 : Démonstration de la réplication et vérification physique

Puisque $RF = 3$, chaque station est dupliquée sur l'ensemble des nœuds (**cass1**, **cass2**, **cass3**).

1. Identifiez un **station_id** présent dans votre table (par exemple **16107**).
2. Vérifiez quels nœuds gèrent physiquement cette station grâce à **nodetool getendpoints**

docker exec -it cass1 nodetool getendpoints velib_cluster stations_velib 16107



```

root@vml3420237:~/TP_cassandra/script# docker exec -it cass1 nodetool getendpoints velib_cluster stations_velib 16107
172.18.0.3
172.18.0.2
172.18.0.4
root@vml3420237:~/TP_cassandra/script#
  
```

La commande liste les adresses **IP** des **3 conteneurs**, confirmant la réplication complète sur le cluster.

Étape 5 : Manipulation des Niveaux de Cohérence (**ONE**, **QUORUM**, **ALL**)

Dans **cqlsh**, testez les différents niveaux de cohérence en lecture :

- **Cohérence ONE** (un seul réplica doit répondre) :

CONSISTENCY ONE;

SELECT * FROM stations_velib LIMIT 1;

```
cqlsh:velib_cluster> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh:velib_cluster> SELECT * FROM stations_velib LIMIT 1;

station_id | bornes_disponibles | capacite | derniere_mise_a_jour | nom_station | vatiques_
disponibles
-----+-----+-----+-----+-----+-----
15047 | 45 | 52 | 2026-10-06 18:25:05.000000+0000 | Morillons - Dantzig |
6

(1 rows)
cqlsh:velib_cluster> 
```

- **Cohérence QUORUM** (majorité des réplicas, soit 2 sur 3) :

CONSISTENCY QUORUM;

SELECT * FROM stations_velib LIMIT 1;

```
(1 rows)
cqlsh:velib_cluster> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:velib_cluster> SELECT * FROM stations_velib LIMIT 1;

station_id | bornes_disponibles | capacite | derniere_mise_a_jour | nom_station | vatiques_
disponibles
-----+-----+-----+-----+-----+-----
15047 | 45 | 52 | 2026-10-06 18:25:05.000000+0000 | Morillons - Dantzig |
6

(1 rows)
cqlsh:velib_cluster> 
```

- **Cohérence ALL** (tous les réplicas doivent répondre) :

CONSISTENCY ALL;

SELECT * FROM stations_velib LIMIT 1;

```

Terminal
cqlsh:velib_cluster> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh:velib_cluster> SELECT * FROM stations_velib LIMIT 1;

 station_id | bornes_disponibles | capacite | derniere_mise_a_jour | nom_station | vatiques_
disponibles
-----+-----+-----+-----+-----+-----
      15047 |          45 |      52 | 2026-10-06 18:25:05.000000+0000 | Morillons - Dantzig |
          6
(1 rows)
cqlsh:velib_cluster> 

```

Étape 6 : Simulation d'une panne

1. Arrêter un nœud (**cass3**) :

docker stop cass3

Exécutez **docker ps** pour confirmer que **cass3** n'est plus actif.

2. Tenter une lecture avec **CONSISTENCY ALL**

```

CONSISTENCY ALL;
SELECT * FROM stations_velib LIMIT 1;

```

La requête lève une **UnavailableException** car 3 réplikas sont requis mais seulement 2 sont en ligne.

3. Insérer une nouvelle donnée avec **CONSISTENCY QUORUM** pendant la panne :

```

CONSISTENCY QUORUM;
INSERT INTO stations_velib (station_id, nom_station, capacite,
bornes_disponibles, vatiques_disponibles, derniere_mise_a_jour)
VALUES ('99999', 'Station Test Panne', 30, 15, 15, toTimestamp(now()));

```

```

Terminal
cqlsh:velib_cluster> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:velib_cluster> INSERT INTO stations_velib (station_id, nom_station, capacite, bornes_disponibles, vatique
s_disponibles, derniere_mise_a_jour)
... VALUES ('99999', 'Station Test Panne', 30, 15, 15, toTimestamp(now()));
cqlsh:velib_cluster> 

```

L'insertion réussit car **QUORUM** (2 réplikas) peut être atteint avec **cass1** et **cass2**

Étape 7 : Rétablissement et consultation sur le nœud redémarré

1. Redémarrer **cass3**

`docker start cass3`

Exécutez **`docker ps`** pour confirmer que **cass3** est de nouveau actif.

2. **Vérifier la resynchronisation directement sur **cass3** :**

Interrogez directement le nœud **cass3** pour vous assurer qu'il a bien récupéré la donnée '99999' grâce au mécanisme de *Hinted Handoff* et de réparation automatique :

`docker exec -it cass3 cqlsh -e "USE velib_cluster; CONSISTENCY ONE; SELECT * FROM stations_velib WHERE station_id = '99999';"`

La **station '99999'** s'affiche correctement, confirmant que le nœud redémarré est parfaitement synchronisé.